

Homework2——Sorting and Paralleling

Task1

考虑到对链表排序的时间复杂度和空间复杂度，我们优先选用Merge Sorting，即归并排序的方式完成。

归并排序即先分治成一块块小部分，最后通过merge函数将两个小部分按顺序拼接。

时间复杂度：

$$O(n \log n)$$

在不考虑递归栈的情况下空间复杂度：

$$O(1)$$

主要实现部分：

```
class ListNode:
    def __init__(self, val=0, next=None):
        self.val=val
        self.next=next

class LLSort:
    def SortList(self, head: ListNode) -> ListNode:
        if not head or not head.next:
            return head

        mid=self.getMiddle(head)
        right_head=mid.next
        mid.next=None

        left=self.SortList(head)
        right=self.SortList(right_head)

        return self.merge(left, right)

    def getMiddle(self, head):
        slow, fast=head, head.next
        while fast and fast.next:
            slow=slow.next
            fast=fast.next.next
        return slow

    def merge(self, l1, l2):
        dummy=ListNode(0)
        curr=dummy
        while l1 and l2:
            if l1.val<=l2.val:
                curr.next=l1
                l1=l1.next
            else:
                curr.next=l2
                l2=l2.next
```

```

        curr=curr.next

    curr.next=l1 if l1 else l2
    return dummy.next

```

生成链表:

```

import random
def CreateLinkedList(n):
    if n==0:
        return None
    head=ListNode(random.randint(-100000,100000))
    curr=head
    for _ in range(1,n):
        curr.next=ListNode(random.randint(-100000,100000))
        curr=curr.next
    return head

```

验证是否成功:

```

def verify(head):
    curr=head
    while curr and curr.next:
        if curr.val>curr.next.val:
            return False
        curr=curr.next
    return True

```

展示链表前n个:

```

import time
def show(head,limit):
    vals=[]
    curr=head
    count=0
    while curr and count<limit:
        vals.append(str(curr.val))
        curr=curr.next
        count+=1
    print("->".join(vals))

```

测试样例:

```

if __name__ == "__main__":
    sol=LLSort()
    head_small=CreateLinkedList(100)
    head_large=CreateLinkedList(10000)
    start_time=time.time()
    sorted_small=sol.SortList(head_small)
    end_time=time.time()

    issorted=verify(sorted_small)

```

```

print(issorted)
print(f"time:{(end_time-start_time)*1000:.20f} ms")
show(sorted_small,20)

start_time=time.time()
sorted_large=sol.SortList(head_large)
end_time=time.time()

issorted=verify(sorted_large)
print(issorted)
print(f"time:{(end_time-start_time)*1000:.6f} ms")
show(sorted_large,20)

```

结果：

```

True
time:0.000000 ms
-98889->-95550->-95199->-93666->-84094->-83316->-82880->-79548->-79471->-79231-
>-71871->-71277->-71175->-69500->-59378->-57124->-56567->-56121->-55108->-53416
True
time:11.999369 ms
-99990->-99933->-99926->-99917->-99886->-99874->-99853->-99847->-99847->-99843-
>-99829->-99769->-99758->-99738->-99727->-99698->-99688->-99684->-99650->-99649

```

遇到的困难包含链表的写法以及递归操作的流程思考；我通过查询和模拟完成归并排序在链表层面的实现。

Task2

GIL(Global Interpreter Lock)

Cpython解释器的互斥锁。保证只有一个线程在CPU上执行代码。对于CPU密集型的任务：若使用ThreadPoolExecutor，线程之间不断切换，会比单线程更慢；对于I/O密集型任务：多线程才能发挥作用。

Thread & Process

多线程仅仅是快速切换任务，受到GIL的限制，虽然共享内存但只能有一个核在作用；

多进程理论上能够利用多核，但不共享内存。故而，由于进程创建和销毁需要开销；数据序列化并通过pipeline发送给子进程，再转回字节流返回主进程的时间开销；以及进程之间的通信延迟开销大，都会共同使看似能加速的并行多进程在数据量比较小的时候开销大、耗时长。

Implementation

在使用python完成多进程操作时，我使用codex生成ProcessPoolExecutor，即多进程操作的代码，与正常串行排序的耗时比较。并且生成了threshold，若超过threshold则并行，不超过则sorted的方式。发现实际上调用多进程的排序耗时远大于串行的排序耗时。

[test.py](#)

Other Sorting Algorithm

其中归并排序、快速排序、桶排序/基数排序比较适合CPU多进程；Bitonic Sort/Odd-Even Sort更适合GPU/FPGA等硬件上的并行计算。

下面仅简述一下在串行情况下，这些排序算法的思路，代码见 [More Sort.ipynb](#)

Quick Sort

使用pivot，把数组分成小于/大于pivot的部分，再对左右两部分递归重复，直到递归的数组长度为1。

Radix Sort

按数字的每一位依次排序，每一位都用计数排序来处理，从低位到高位逐轮排序，最终所有位排完数组就整体有序。

Bucket Sort

将待排序元素分到有限数量的有序桶里，每个桶内单独排序，最后按桶的顺序把所有元素依次取出，就得到完整有序数组。

其中，每个桶里面用快排！

Bitonic Sort

先把数组构造造成“先升后降 / 先降后升”的双调序列，再不断合并成有序序列。

Odd-even Sort

通过交替比较交换偶数索引对、奇数索引对，逐步让元素归位的并行冒泡排序。

困难就是无法实际完成并行计算，在小数据的情况下效果很一般，只能理解上述五种算法的实现原理，知道他适合做并行计算。